Sistemi Operativi - Manuale Completo

Processi

Il kernel mantiene le informazioni sul processo in una struttura dati detta **PCB** (Process Control Block), questa viene aggiornata ad ogni context switch ed usata per caricare il nuovo processo.

Il passaggio della CPU da un processo all'altro viene detto **context switch**: + sospensione processo in esecuzione, aggiornamento PCB + caricamento nuovo processo, tramite PCB

Unix syscall

Un processo può creare numerosi processi figli tramite la chiamata di sistema `fork()`, restituisce:

- `= 0` al processo figlio
- `= pid figlio <> 0` al processo padre

Alla creazione il figlio eredita dal padre:

- copia della memoria
- registri CPU
- tutti i file aperti

Sarà identico alla creazione il PCB.

Usiamo la chiamata di sistema `exec()` per caricare un nuovo programma.

La terminazione normale di un processo avviene tramite la chiamata `exit(status)` dopo l'ultima istruzione, cosa fa:

- eseguite le funzioni passate ad `atexit(fun)` ed `on_exit(fun)`
- tutte le stream sono scaricate con `fflush()`
- chiusi tutti i file e connessioni aperte, tranne quelli condivisi con altri processi
- chiama `_exit(status)`

La syscall `_exit(status)` cosa fa:

- salva argomento d'uscita
- dealloca risorse processo
- assegna eventuali figli zombie ad init
- se ha un padre diverso da init in esecuzione diventa zombie per conservare l'argomento fino ad attesa, altrimenti termina definitivamente

L'esecuzione del padre dopo la generazione di un figlio può continuare concorrentemente oppure attendere la terminazione con la chiamata di sistema `wait()`.

La chiamata `wait(NULL)` blocca il padre fino alla terminazione di un qualunque figlio, invece `waitpid(..., int pid, ...)` permette di attendere la terminazione di un specifico figlio passando il pid.

Il padre riceve il valore di ritorno aspettando la terminazione del figlio.

```c
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

Il padre può terminare l'esecuzione del figlio tramite interruzioni, detto aborto.

Se il padre termina prima del figlio oppure non attende la terminazione si genera un processo **zombie** che viene adottato dal processo **init** ed eseguito fino alla terminazione.

Non UNIX ma altri SO implementano la terminazione a cascata, alla terminazione di processo non permettono la continuazione dei suoi figli.

Thread

Detto anche lightweight process è l'unità di esecuzione/scheduling del processo, indipendente dall'unità di gestione delle risorse detta task.

Il tradizionale processo monothread esegue un'attività alla volta in modo sequenziale, ogni fork implica:

- risorse di memoria duplicate
- addizionale scheduling e context switch

La soluzione è la condivisione delle risorse fra attività analoghe, un thread è un flusso di esecuzione indipendente all'interno dello stesso processo.

Tra i thread interni al processo sono condivisi:

- spazio di indirizzamento
- scheduling tramite TCB che punta al PCB di appartenenza comune

Il **Thread Control Block** è in SCL con altri del processo e contiene:

- program counter
- stato computazione
- descrittori di thread
- memoria privata, o **thread specific data** (TSD)

Ogni thread ha uno stato di esecuzione e si può sincronizzare con gli altri thread all'interno dello stesso processo.

Su sistemi single-core l'esecuzione concorrente è consentita solo grazie ad **interleaving** tra i thread, invece sistemi multi-core permettono l'effettiva esecuzione concorrente dei thread.

In caso di architettura multi-core bisogna progettare adeguatamente:

- i SO devono fornire schedulatori multi-core
- nelle applicazioni bisogna tenere in conto:
  - separazione e bilanciamento dei task per esecuzione in parallelo
  - suddivisione dati ed accesso sincronizzato (thread-safe)
  - test e debugging più complessi

A seconda del livello di processo a cui appartiene il thread abbiamo:

- **user-level thread**: appartenenti a processi user-level sono gestiti da apposite librerie e lo switch non richiede chiamate al kernel
- **kernel-level thread**: appartenenti a processi kernel-level sono gestiti con chiamate di sistema ed interamente dal kernel, detti kernel-supported threads

Questi si combinano per consentire una comunicazione multi-threading a due vie e permettere l'accesso alle risorse:

- **molti a uno**: ma se uno fa chiamata bloccante sono tutti bloccati, impossibile thread in parallelo
- **uno a uno**: mappa ogni thread, possibile thread in parallelo anche con chiamate bloccanti
- **molti a molti**: rende personalizzabile il numero massimo di thread in parallelo
- **a due livelli**: rende possibile sia molti a molti che uno a uno

L'interruzione del thread prima che termini l'esecuzione può essere:

- **asincrona**: se un altro thread termina immediatamente il target, possono rimanere non libere le risorse
- **differita**: se il target controlla periodicamente se deve terminare così da eventualmente liberare le risorse e poi terminare

Si dice **thread pool** un insieme di thread a cui il processo può assegnare task ed alla terminazione tornano disponibili nell'insieme.

Tra thread kernel e thread utente è posta una struttura dati detta **LightWeight Process** (LWP), vista dalla libreria del thread utente come un processore virtuale su cui schedulare i thread utente, questo LWP è dipendente dall'esecuzione del thread kernel associato.

Così i thread kernel possono comunicare eventi alla libreria dei thread utenti con le chiamate ai thread dette **upcall**.

Stati dei Thread

Gli stati dei thread sono:

- **running**: in esecuzione
- **ready**: pronto per l'esecuzione
- **blocked**: bloccato in attesa di risorsa

Le operazioni di transizione sono:

- **spawn**: creazione del thread
- **block/unblock**: blocco e sblocco
- **finish**: terminazione

Concorrenza

Diamo alcune definizioni tecniche della concorrenza nei SO.

Una **race condition** si verifica quando due o più processi/thread competono per l'accesso ad una risorsa condivisa, ed il risultato dell'elaborazione dipende dall'ordine di accesso. Comporta comportamenti non deterministici difficili da diagnosticare.

La **mutual exclusion** è il vincolo che impone che in qualsiasi momento un solo processo/thread possa accedere alla risorsa condivisa, questo risolve il problema della race condition e riduce il rischio di inconsistenza dei dati.

Con **bounded waiting** richiede che in ogni implementazione di mutua esclusione nessun processo possa aspettare indefinitamente il proprio turno.

La **critical section** è la parte di codice la cui esecuzione contiene l'accesso alla risorsa condivisa, serve implementare mutua esclusione.

Il **deadlock** è la situazione di stallo in cui due o più processi/thread rimangono bloccati in attesa del rilascio di una risorsa condivisa da un altro, creando un circolo vizioso di cui nessun partecipante avanza.

Il **livelock** è la situazione di stallo attivo simile al deadlock, in cui i processi/thread coinvolti non sono bloccati in attesa ma in un loop infinito di transizioni senza fare alcun progresso.

La **starvation** è la situazione in cui uno o più processi/thread sono bloccati per un tempo indefinito in attesa di accedere ad una risorsa condivisa, altre operazioni continuano ad ottenere la priorità.

Una **operazione atomica** è un'azione o funzione che viene eseguita come unità indivisibile, garantisce che venga completata integralmente o per niente. Nessun processo/thread esterno può osservare stati intermedi o interrompere l'operazione mentre è in corso.

Un processo/thread può richiedere la **disabilitazione degli interrupt** mentre è nella sezione critica per evitare accessi concorrenti, su sistemi multi-core è inefficace visto che disabilita interrupt su un solo core alla volta.

Alcune architetture hardware supportano **istruzioni HW speciali** che garantiscono l'atomicità per operazioni.

Busy Waiting

La mutua esclusione si può realizzare con la tecnica del **busy waiting**, consiste nell'usare un ciclo infinito per controllare la condizione desiderata di accesso alla risorsa. Il processo/thread usa la CPU continuamente e potrebbe bloccarne altri.

Si usa una **variabile di turno** come flag che vale 0 se la risorsa è libera oppure 1 se è in uso. I processi interessati controllano continuamente, quando vale 0 accedono ed eseguono la sezione critica.

Semafori

Sono una delle strutture di sincronizzazione più comuni per gestire l'accesso a risorse condivise tra thread, garantendo la mutua esclusione usano primitive HW.

I semafori **contatori** servono per tracciare la disponibilità di un certo numero di risorse, assume valori non negativi. Se 0 allora non ci sono risorse libere, altrimenti ne indica il numero.

I semafori **binari** servono per garantire la mutua esclusione di due o più thread su una risorsa condivisa. Se 0 la risorsa è in uso, se 1 la risorsa è libera.

Il **mutex** è un particolare semaforo binario usato per mutua esclusione tra threads. Solo il thread che ha bloccato il mutex può rilasciarlo.

Abbiamo due diverse tipologie di gestione della coda di priorità per semafori:

- gli **strong semaphores** implementano una coda FIFO
- i **weak semaphores** non specificano l'algoritmo di rimozione dalla coda

Assumiamo solo strong semaphores.

I **named semaphores** sono la struttura di sincronizzazione comune per gestire l'accesso a risorse condivise tra processi. Il semaforo viene creato con un nome univoco nel file system e ci si riferisce tramite il nome.

Le operazioni supportate dai semafori sono:

- `sem_init()` per inizializzare un semaforo con un valore
- `sem_wait()` se il semaforo ha valore positivo lo decrementa all'accesso in risorsa, altrimenti blocca il chiamante in attesa
- `sem_post()` incrementa il valore del semaforo al rilascio della risorsa

Altri meccanismi mutua esclusione

Un **monitor** è un meccanismo di sincronizzazione che incapsula sia i dati condivisi sia le funzioni che possono operarvi, garantisce che un solo thread alla volta vi acceda. Può avere una coda di attesa.

Gli **event flags** vengono usati per notificare delle condizioni tra thread concorrenti, utile quando uno deve aspettare l'esecuzione di un'operazione da parte di qualcun altro.

Le **mailbox** sono meccanismi di comunicazione tra thread, ognuno possiede un buffer per la ricezione e può scrivere nei buffer altrui.

Una **barrier** si usa per sincronizzare più thread in un punto specifico del programma, garantisce che tutti si attenda l'arrivo di tutti al checkpoint.

Lo **spinlock** è utile per brevi sezioni critiche, garantisce mutua esclusione concedendo il lock ad un solo thread alla volta, mettendo in spin gli altri. Utile solo su multi-core in quanto blocca la CPU.

Algoritmo Producer-Consumer

Due tipi di processi che interagiscono:

- i **producer** generano dati e li scrivono in un buffer condiviso
- i **consumer** leggono ed elaborano i dati nel buffer condiviso

Bisogna garantire che: + nessun producer scriva quando il buffer è pieno + nessun consumer legga quando il buffer è vuoto + mutua esclusione tra scrittura e lettura

Il producer genera un item e verifica la disponibilità di spazio del buffer, se è pieno attende che si liberi. Ottenuto lo spazio scrive e segnala la disponibilità dei dati.

Il consumer desidera elaborare e verifica la disponibilità di dati del buffer, se è vuoto attende che siano scritti dati. Rilevati dati vengono letti e segnalato il consumo.

```
/* BOUNDED BUFFER */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n = 0, e = sizeofbuffer;

void producer()
{
  while (true)
  {
    produce();
    semWait(e);   // richiesta spazio
    semWait(s);   // accesso mutex cs
    append();
    semSignal(s); // rilascio mutex cs
    semSignal(n); // notifica dati
  }
}

void consumer()
{
  while (true)
  {
    semWait(n);   // richiesta dati
    semWait(s);   // accesso mutex cs
    take();
    semSignal(s); // rilascio mutex cs
    semSignal(e); // notifica spazio
    consume();
  }
}
```

Dove il semaforo binario **s** gestisce la mutua esclusione sulla sezione critica, i semafori contatori **n**, **e** indicano il numero di dati da consumare ed il numero di slot liberi nel buffer.

Algoritmo Reader-Writer

Problema di accesso concorrente ad una risorsa condivisa da più processi che possono leggere o scrivere dati, avremo lettori e scrittori.

Bisogna garantire che:

- più lettori possano accedere contemporaneamente
- solo uno scrittore alla volta
- durante la scrittura nessun lettore può accedere

Il reader accede se nessuno sta scrivendo, se non vi sono già altri reader notifica al sistema l'inizio della lettura bloccando l'accesso ai writer. Se sono presenti già altri reader entra liberamente senza attesa.

Il writer per accedere deve assicurarsi che:

- non vi siano reader attivi
- non vi siano altri writer attivi
- entri da solo, bloccando accesso ad ogni altro reader o writer

Useremo semafori per segnalare quanti lettori sono attivi e se uno scrittore è in attesa, possiede due varianti a seconda dell'operazione con priorità:

- la **priorità ai reader** impedisce ai writer di ottenere priorità in caso di continuo accesso in lettura, possibile starvation dei writer
- la **priorità ai writer** impedisce ai reader di ottenere priorità in caso di continuo accesso in scrittura, possibile starvation dei reader

```
/* WRITERS PRIORITY */
int readcount = 0, writecount = 0;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;

void reader()
{
  while (true)
  {
    semWait(z);                             // attesa in coda z
    semWait(rsem);                          // primo reader in coda su rsem
    semWait(x);                             // incrementa readcount
    readcount++;
    if (readcount == 1) semWait(wsem);      // blocca i writer
    semSignal(x);
    semSignal(rsem);                        // scorre coda reader
    semSignal(z);                           // scorre coda z
    READUNIT();                             // la lettura può avvenire concorrente
    semWait(x);                             // decrementa readcount
    readcount--;
    if (readcount == 0) semSignal(wsem);    // sblocca i writer se finiti reader
    semSignal(x);
  }
}

void writer()
{
  while (true)
  {
    semWait(y);                             // incrementa writecount
    writecount++;
    if (writecount == 1) semWait(rsem);     // blocca i reader
```

```
    semSignal(y);
    semWait(wsem);                          // accede sezione critica
    WRITEUNIT();
    semSignal(wsem);                        // libera sezione critica
    semWait(y);                             // decrementa writecount
    writecount--;
    if (writecount == 0) semSignal(rsem); // sblocca i reader
    semSignal(y);
  }
}
```

Dove:

- **x** gestisce la variabile condivisa *readcount* che conta il numero di lettori in coda
- **y** gestisce la variabile condivisa *writecount* che conta il numero di scrittori in coda
- **z** accoglie coda *reader* permettendo ad un solo lettore di attendere su *rsem*, garantendo priorità agli scrittori
- **rsem** è il semaforo mutex lettura, accoglie un solo lettore attivo
- **wsem** è il semaforo mutex scrittura, accoglie coda *writers*

Deadlock

Il deadlock è una situazione di stallo in cui due o più processi restano bloccati in modo permanente, in quanto ognuno richiede una risorsa in uso da un altro.

Risorse

Le risorse **riusabili** possono essere usate più volte da diversi processi senza essere alterate o consumate. Una volta rilasciate possono essere immediatamente riassegnate.

Le risorse **consumabili** vengono esaurite o distrutte dopo essere state usate da un processo. Per essere riassegnata deve essere ricreata o rigenerata.

Condizioni necessarie per il deadlock

Le condizioni in cui si verifica sono:

- **mutua esclusione** sulle risorse
- **hold and wait** di risorse
- **no preemption**, ovvero non si può forzare il rilascio di una risorsa da un altro processo
- **circular waiting**, una catena di risorse richieste e possedute dai processi coinvolti

Prevenzione del deadlock

Necessario che almeno una delle 4 condizioni sopra non si verifichi per prevenire il deadlock:

- la **mutua esclusione** non si può rimuovere ma minimizzarne l'uso
- **hold and wait** ha una inefficace soluzione facendo richiedere ed attendere la disponibilità di tutte le risorse prima di iniziare
- si forza la **preemption** distribuendo le risorse in modo equo, solo su risorse salvabili e ripristinabili
- l'**attesa circolare** si elimina forzando la richiesta delle risorse in un preciso ordine, potrebbe essere inefficiente per il programma

Deadlock Avoidance

Un sistema si dice in **stato sicuro** (*deadlock-safe state*) se nel caso in cui tutti i processi richiedessero il massimo delle risorse, fosse in grado di soddisfarle senza entrare in deadlock.

Resource Allocation Denial Il SO controlla se allocando le risorse richieste rimarrebbe in uno stato sicuro, altrimenti rifiuta la richiesta.

Process Initiation Denial Il SO controlla se le risorse richieste da un processo sono disponibili prima di inizializzarlo, altrimenti non lo permette.

Algoritmo del Banchiere L'algoritmo del banchiere è un approccio di deadlock avoidance nei sistemi di gestione delle risorse, assicurandosi che ogni stato sia sicuro per l'esecuzione di processi. Il sistema deve essere in grado di soddisfare le richieste di tutti i processi senza finire in deadlock.

```
boolean safe (state S)
{
  int currentavail[m];
  process rest[<number of processes>];
  currentavail = available;       // le risorse disponibili
  rest = {all processes};         // processi che devono terminare
  possible = true;                // flag se il processo può proseguire

  while (possible)
  {
    // cerchiamo il processo che richiede risorse aggiuntive
    // minori delle disponibili
    <find Pk in rest such that claim[k,*] - alloc[k,*] <= currentavail;>

    if (found) {
      /* simulate pk execution */
      currentavail = currentavail + alloc[k,*];   // risorse di pk rilasciate
      rest = rest - {pk};                         // rimosso pk che ha completato
    }
    else possible = false;       // nessun processo può proseguire
  }
  return (rest == null);         // true = stato sicuro, false = pericolo deadlock
}
```

In sintesi verifica se esiste una sequenza di esecuzione dei processi che presenta rischio deadlock.

Deadlock Detection

La deadline detection si riferisce alla capacità del sistema di monitorare e rilevare quando un'attività non riesce a completarsi entro il suo limite di tempo.

Il sistema operativo utilizza algoritmi di schedulazione che tengono conto delle deadline delle attività. Questi algoritmi tentano di ordinare le attività in modo da garantire che quelle con deadline più ravvicinate vengano eseguite per prime, riducendo il rischio di mancato rispetto delle scadenze.

Quando una deadline non viene rispettata il sistema notifica i processi correlati e può adottare due soluzioni:

- **terminazione** dei processi che superano la deadline
- **ripristino** dei processi che superano la deadline ad un checkpoint definito prima

Problema dei Dining Philosophers

Classico problema di sincronizzazione e gestione di risorse condivise in sistema multi-thread.

Le risorse concorrenti sono le forchette ed i processi sono i filosofi.

> 5 filosofi sono seduti a cena e possono fare solo una di due cose: mangiare o pensare. Tra ogni coppia di filosofi adiacenti c'è una forchetta. Per mangiare ogni filosofo ha bisogno della forchetta destra e sinistra, tuttavia ce ne sono solo 5. Se tutti i filosofi prendono la forchetta a sinistra, nessuno può prendere la destra finendo in deadlock (e viceversa).

Il problema si risolve ponendo un semaforo per forchetta, così che possa essere detenuta da uno solo. Così generiamo però *hold and wait* e *circular waiting*, quindi deadlock essendo assunte *no-preemption* e *mutua esclusione* sulle forchette. Serve un ulteriore semaforo **room** che limita il numero di filosofi a 4, così almeno uno alla volta potrà mangiare.

```
semaphore fork[5] = {1};          // tutte le forchette sono disponibili
semaphore room = {4};             // max 4 filosofi nella stanza

void philosopher (int i)
{
  while (true)
  {
    think();
    semWait(room);                // verifica disponibilità stanza
    semWait(fork[i]);             // verifica disponibilità forchetta sx
    semWait(fork[(i+1) % 5]);     // verifica disponibilità forchetta dx
    eat();
    semPost(fork[(i+1) % 5]);     // rilascio forchetta dx
    semPost(fork[i]);             // rilascio forchetta sx
    semPost(room);                // rilascio stanza
  }
}
```

Algoritmo di Dekker

Uno dei primi algoritmi di mutua esclusione per gestire la concorrenza tra due processi su risorsa critica condivisa, esclusivamente via software.

```
/* global */
int turn = {0,1};
boolean flag[2] = {false, false};

/* processo P0 */
int me = 0, other = 1;          // pov P0

while (true)
{
  /* NCS */
  flag[me] = true;              // richiesta turno
  while (flag[other])           // se ha richiesta anche P1
  {
    if (turn == other)          // se è già il turno di P1
    {
      flag[me] = false;         // rinuncio temporaneamente al turno
      while (turn == other);    // busy waiting fine turno P1
      flag[me] = true;          // ottengo il turno
    }
  }
  /* CS */
  turn = other;                 // rilascio il turno
  flag[me] = false;             // reset flag di richiesta
}
```

La variabile globale `int turn` determina la priorità tra processi. L'array globale `boolean flag[2]` rappresenta l'interesse dei processi alla sezione critica. Le variabili locali `me` e `other` identificano i processi: P0 vede `me=0` e `other=1` e viceversa per P1.

Prima della sezione critica il processo:

- imposta `flag[me]=true` per dichiarare interesse
- verifica se l'altro è già interessato con `while (flag[other])`, altrimenti accede
- se `flag[other]=true` ed il turno è dell'altro busy waiting con `while (turn == other)`, altrimenti accede
- dopo l'accesso in sezione critica
  - imposta il turno all'altro con `turn = other`
  - rilascia interesse con `flag[me] = false`

Algoritmo di Dijkstra

Generalizzazione dell'algoritmo di Dekker per N processi, garantisce assenza di deadlock e starvation.

```
/* global */
boolean interested[N] = {false};
```

```
boolean passed[N] = {false};
int k;                          // arbitrario

/* processo Pi */
int i;                          // id processo locale

while (true)
{
  /* NCS */
  interested[i] = true;         // dichiaro interesse

  while (k != i)                // se non ho precedenza
  {
    passed[i] = false;          // segnalo attesa
    if (!interested[k])         // se k non interessato
      k = i;                    // prendo il suo posto
  }

  passed[i] = true;             // ho precedenza

  for (int j = 0; j < N; j++)   // controllo che nessun altro abbia passato
  {
    if (j != i && passed[j])
      goto step2;               // torno al while
  }

  /* CS */
  interested[i] = false;        // rilascio interesse
  passed[i] = false;            // rilascio controllo
}
```

Algoritmo del Barbiere Addormentato

Problema di produttore-consumatore con vincoli di sincronizzazione.

Una barberia con:

- Un barbiere che serve un cliente alla volta
- Una poltrona del barbiere
- N sedie d'attesa
- Clienti che arrivano per servizio

Regole:

- se non ci sono clienti, il barbiere dorme
- se arriva cliente e barbiere dorme, lo sveglia
- se barbiere occupato, cliente si siede in attesa (se c'è posto)
- se sedie piene, cliente se ne va

```
/* global */
semaphore waitingCustomers = 0;   // clienti in attesa
semaphore barberReady = 0;        // barbiere pronto
```

```
semaphore accessSeats = 1;          // mutex sedie
int waitingSeats = 0;               // sedie occupate
const int MAX_CHAIRS = N;

void customer()
{
  wait(accessSeats);                // controllo sedie

  if (waitingSeats < MAX_CHAIRS)    // se c'è posto
  {
    waitingSeats++;                 // occupo sedia
    signal(waitingCustomers);       // avviso barbiere
    signal(accessSeats);            // rilascio mutex
    wait(barberReady);              // attendo barbiere
    get_haircut();                  // mi faccio servire
  }
  else                              // niente posto
  {
    signal(accessSeats);            // rilascio mutex
    // cliente se ne va
  }
}

void barber()
{
  while (true)
  {
    wait(waitingCustomers);         // attendo cliente
    wait(accessSeats);              // controllo sedie
    waitingSeats--;                 // libero sedia
    signal(barberReady);            // segnalo pronto
    signal(accessSeats);            // rilascio mutex
    cut_hair();                     // servo cliente
  }
}
```

Algoritmo della Bakery

Algoritmo per mutua esclusione nei sistemi concorrenti quando più processi devono accedere a risorsa condivisa.

Funziona come panetteria dove clienti prendono numero per mettersi in coda, viene servito chi ha numero più basso.

```
/* global */
boolean choosing[N] = {false};
int number[N] = {0};

/* processo Pi */
int i;
```

```
while (true)
{
  /* NCS */

  // Fase 1: scelgo numero
  choosing[i] = true;
  number[i] = max(number[0], ..., number[N-1]) + 1;
  choosing[i] = false;

  // Fase 2: attendo il mio turno
  for (int j = 0; j < N; j++)
  {
    while (choosing[j]);          // attendo che j scelga

    // attendo se j ha numero minore o stesso numero ma indice minore
    while (number[j] != 0 &&
           (number[j] < number[i] ||
           (number[j] == number[i] && j < i)));
  }

  /* CS */

  number[i] = 0;                  // rilascio turno
}
```

Inter-Process Communication (IPC)

La comunicazione inter-processo è fondamentale per permettere ai processi di scambiarsi informazioni e collaborare.

Categorie di IPC

Esistono due categorie principali:

1. Trasferimento Dati (Data Transfer) I dati vengono trasmessi da un processo all'altro tramite canale di comunicazione fornito dal SO:

- **Segnali (Signals)**: meccanismo semplice per notificare eventi, trasportano informazioni limitate
- **Pipe**: canali unidirezionali per processi correlati, flusso continuo FIFO
- **Socket**: comunicazione bidirezionale anche tra macchine diverse in rete
- **Code di messaggi (Message Queue)**: comunicazione asincrona con messaggi strutturati

2. Memoria Condivisa (Shared Memory) Regione di memoria accessibile a più processi per lettura/scrittura diretta.

- metodo più veloce per IPC
- richiede sincronizzazione (semafori, mutex)

- rischio conflitti e corruzione dati
- kernel gestisce creazione, processi accedono come memoria ordinaria

Pipe

Canale di comunicazione unidirezionale tra processi correlati, implementa il message passing.

Caratteristiche

- buffer gestito dal SO, dimensione fissa (tipicamente 4096 byte)
- FIFO (First In, First Out)
- creato tramite chiamata di sistema `pipe(int fd[2])`
  - `fd[0]`: descrittore per lettura
  - `fd[1]`: descrittore per scrittura
- processo scrive su `fd[1]`, altro processo legge da `fd[0]`

Comportamento

- **Scrittura su pipe piena**: processo bloccato fino a spazio disponibile
- **Lettura da pipe vuota**: processo bloccato fino a dati disponibili
- **Chiusura scrittori**: lettori ricevono EOF
- **Chiusura lettori**: scrittori ricevono segnale SIGPIPE

Atomicità

- Scritture <= PIPE_BUF (4096 byte) sono atomiche
- Scritture > PIPE_BUF possono essere intercalate

```c
#include <unistd.h>

int fd[2];
pipe(fd);                    // crea pipe

// processo figlio legge
close(fd[1]);                // chiude lato scrittura
read(fd[0], buffer, size);   // legge dati

// processo padre scrive
close(fd[0]);                // chiude lato lettura
write(fd[1], data, size);    // scrive dati
```

Named Pipe (FIFO)

Pipe con nome nel filesystem, permette comunicazione tra processi non correlati.

Caratteristiche

- persiste nel filesystem anche dopo terminazione processi
- accessibile tramite nome comune
- rimossa con `unlink()`
- supporta apertura con flag O_NONBLOCK

```c
#include <sys/stat.h>
#include <fcntl.h>

// creazione FIFO
mkfifo("/tmp/myfifo", 0666);

// apertura per scrittura
int fd = open("/tmp/myfifo", O_WRONLY);
write(fd, data, size);
close(fd);

// apertura per lettura
int fd = open("/tmp/myfifo", O_RDONLY);
read(fd, buffer, size);
close(fd);

// rimozione
unlink("/tmp/myfifo");
```

Socket

Meccanismo di comunicazione bidirezionale tra processi, anche su macchine diverse.

Tipi di Socket **Stream Socket (TCP)**:

- orientati alla connessione
- affidabili, ordinati
- controllo flusso e ritrasmissione
- overhead maggiore

Datagram Socket (UDP):

- non orientati alla connessione
- non affidabili, nessun ordine garantito
- basso overhead, latenza minore
- adatto per streaming, gaming

Raw Socket:

- bypassano livello trasporto
- accesso diretto a protocolli di rete

Identificazione Socket Combinazione di:

- **Indirizzo IP**: identifica dispositivo sulla rete
- **Numero Porta**: identifica applicazione/servizio sul dispositivo

Porte

- **Well-Known Ports (0-1023)**: servizi standard
  - 80: HTTP
  - 443: HTTPS

  - 25: SMTP
  - 53: DNS
- **Registered Ports (1024-49151)**: applicazioni registrate
  - 3306: MySQL
- **Dynamic/Private Ports (49152-65535)**: assegnate dinamicamente

---

Scheduling

Lo scheduling è la funzione del SO che decide quale processo eseguire sulla CPU e per quanto tempo.

Tipi di Scheduler

Short-Term Scheduler (CPU Scheduler)

- determina quale processo ready in RAM deve essere eseguito
- eseguito molto frequentemente (millisecondi)
- deve essere estremamente rapido
- obiettivo: ottimizzare uso CPU

Medium-Term Scheduler

- gestisce swapping tra RAM e disco
- sospende processi (swap-out) e li ripristina (swap-in)
- frequenza intermedia
- obiettivo: ottimizzare uso memoria, ridurre congestione

Long-Term Scheduler (Job Scheduler)

- seleziona processi da caricare in memoria dalla coda job
- eseguito meno frequentemente
- obiettivo: determinare grado multiprogrammazione

Stati dei Processi

- **New**: processo creato, non ancora in ready
- **Ready**: pronto per esecuzione, attende CPU
- **Running**: in esecuzione sulla CPU
- **Wait/Block**: attende risorsa o I/O
- **Suspended Ready**: pronto ma swapped out
- **Blocked Suspended**: bloccato e swapped out
- **Exit**: terminato, rimosso dal sistema

Criteri di Ottimizzazione

Criterio	Definizione	Obiettivo
Throughput	Numero processi completati per unità tempo	Massimizzare

Criterio	Definizione	Obiettivo
Response Time	Tempo tra sottomissione e prima risposta	Minimizzare
Waiting Time	Tempo totale in coda ready	Minimizzare
Turnaround Time	Tempo totale da immissione a completamento	Minimizzare
CPU Usage	Percentuale tempo CPU attiva	Massimizzare

Context Switch

Operazione di cambio processo sulla CPU:

Fasi:

1. **Sospensione processo corrente**: trigger da preemption, I/O o scadenze
2. **Salvataggio stato nel PCB**: registri, PC, stack pointer, info memoria
3. **Spostamento PCB in coda**: ready o waiting
4. **Ripristino nuovo processo**: carica stato da PCB

Il tempo di context switch è overhead che non contribuisce al lavoro utile.

Algoritmi di Scheduling

Sistemi Batch First-Come First-Served (FCFS):

- non preemptive
- processi serviti in ordine arrivo
- semplice ma può causare convoy effect

Shortest Job First (SJF):

- non preemptive
- seleziona processo con minor tempo esecuzione
- minimizza tempo medio attesa
- difficile stimare tempo esecuzione

Shortest Remaining Time Next (SRTN):

- preemptive, variante di SJF
- seleziona processo con minor tempo rimanente
- può interrompere processo corrente
- minimizza attesa ma overhead per preemption

Sistemi Interattivi Round Robin (RR):

- preemptive
- time slice fissa per ogni processo
- coda circolare FIFO
- equo ma dipende da dimensione time slice
  - troppo piccola: overhead context switch
  - troppo grande: comportamento simile a FCFS

Priority Scheduling:

- ogni processo ha priorità
- processi alta priorità eseguiti prima
- due varianti: preemptive e non-preemptive
- rischio starvation per basse priorità

Multilevel Queue:

- processi divisi in code con priorità diversa
- ogni coda può avere proprio algoritmo
- code servite in ordine priorità
- rischio starvation code basse

Shortest Process Next:

- analogo a SJF per sistemi interattivi
- non preemptive

Highest Response Ratio Next (HRRN):

- non preemptive
- ratio = (Tempo Attesa + Tempo Servizio) / Tempo Servizio
- bilancia equità ed efficienza
- riduce starvation

Guaranteed (Fair) Scheduling:

- ogni utente riceve 1/n del tempo CPU (n = numero utenti)
- ratio = Tempo Usato / Tempo Autorizzato
- esegue processo con ratio più basso

Calcolo Metriche

Dati:

- Processi: P_1, …, P_n
- Tempi Arrivo: A_1, …, A_n
- Tempi Servizio: S_1, …, S_n
- Tempi Completamento: C_1, …, C_n

Throughput:

```
n / (max(C_1,...,C_n) - min(A_1,...,A_n))
```

Turnaround Time medio:

```
Σ_i(C_i - A_i) / n
```

Waiting Time medio:

```
Σ_i(C_i - A_i - S_i) / n
```

Normalized Turnaround medio:

```
Σ_i((C_i - A_i) / S_i) / n
```

Efficienza algoritmo:

- Rapporto = Turnaround Time / Service Time
- = 1: processo non ha mai atteso (ottimale)

- 1: processo ha atteso (quanto più alto, peggio)

Gestione della Memoria

Il memory manager è responsabile dell'allocazione e controllo della memoria principale (RAM).

Funzioni del Memory Manager

- **Allocazione**: riserva memoria ai processi
- **Deallocazione**: libera memoria non più necessaria
- **Monitoraggio**: traccia uso memoria
- **Swapping**: sposta processi tra RAM e disco
- **Protezione**: impedisce accessi non autorizzati

Tecniche di Tracciamento

Bitmap

- sequenza di bit rappresenta stato memoria
- 0 = blocco libero, 1 = blocco occupato
- compatta ma ricerca sequenziale lenta
- inefficiente per grandi allocazioni

Liste Collegate

- ogni elemento rappresenta segmento memoria
- contiene: tipo (processo/libero), indirizzo inizio, lunghezza
- permette unione efficiente segmenti adiacenti liberi
- utile per processi dimensioni variabili

Algoritmi di Allocazione

First-Fit:

- assegna primo blocco abbastanza grande
- veloce ma può causare frammentazione

Best-Fit:

- assegna blocco più piccolo che soddisfa richiesta
- riduce frammentazione esterna
- più lento, ricerca completa

Worst-Fit:

- assegna blocco più grande disponibile
- generalmente inefficace

Next-Fit:

- come First-Fit ma inizia da ultima allocazione
- riduce tempo ricerca

Frammentazione

Frammentazione Interna:

- memoria allocata ma non utilizzata completamente
- blocco più grande del necessario
- spreco risorse

Frammentazione Esterna:

- memoria libera frammentata in piccoli blocchi
- somma sufficiente ma non contigua
- impedisce allocazioni grandi

Memoria Virtuale

Permette ai processi di usare spazio indirizzi più ampio della RAM fisica.

Vantaggi

- ogni processo ha proprio spazio virtuale
- isolamento e sicurezza tra processi
- possibile eseguire programmi più grandi della RAM

Paginazione

Tecnica che divide memoria in blocchi dimensione fissa.

Caratteristiche

- **Pagine**: blocchi memoria virtuale (dimensione fissa)
- **Frame**: blocchi memoria fisica (dimensione = pagina)
- elimina frammentazione esterna
- possibile frammentazione interna (ultima pagina)
- memoria virtuale contigua, fisica potrebbe non esserlo

Dimensione Pagine

- **Piccole**: meno frammentazione interna, più overhead tabella
- **Grandi**: più frammentazione interna, meno overhead tabella
- dimensioni troppo piccole: più page fault
- dimensioni troppo grandi: rischio thrashing

Demand Paging

- paginazione su richiesta (lazy)
- pagine caricate in RAM solo quando accedute
- riduce uso memoria e swapping
- tutte pagine inizialmente su disco

Tabella delle Pagine Struttura dati che mappa pagine virtuali a frame fisici.

Contenuto entry:

- numero pagina virtuale
- numero frame fisico
- flag controllo (permessi, modificata, presente)

Traduzione indirizzo:

- indirizzo virtuale = [numero pagina | offset]
- numero pagina → consulta tabella → numero frame
- indirizzo fisico = [numero frame | offset]

Translation Lookaside Buffer (TLB)

- cache hardware per traduzioni recenti
- parte della MMU (Memory Management Unit)
- velocizza traduzione indirizzi
- TLB hit: traduzione immediata
- TLB miss: consulta tabella pagine, aggiorna TLB

Tabella Pagine Invertita

- una entry per frame fisico (non per pagina)
- contiene: indirizzo pagina virtuale, PID processo
- riduce uso memoria per tabella
- ricerca tramite hashtable

Tabella Pagine a Due Livelli

- divide tabella pagine in due livelli
- indirizzo virtuale = [indice liv1 | indice liv2 | offset]
- riduce memoria necessaria
- alloca solo tabelle necessarie

Segmentazione

Divide memoria in segmenti logici dimensione variabile.

Segmenti tipici:

- **Code**: istruzioni eseguibili
- **Data**: variabili e strutture dati
- **Stack**: chiamate funzioni e variabili locali
- **Heap**: memoria dinamica

Può essere combinata con paginazione: segmenti divisi in pagine.

Swapping

Trasferimento processi/pagine tra RAM e disco.

Page Fault

1. Processo accede pagina non in RAM
2. Se RAM piena, seleziona pagina vittima
3. Se pagina vittima modificata, salva su disco (swap-out)
4. Carica pagina richiesta da disco (swap-in)
5. Aggiorna tabella pagine
6. Riprende esecuzione

Algoritmi Sostituzione Pagine Optimal (OPT):

- teorico, non implementabile
- sostituisce pagina usata più lontano nel futuro
- minimizza page fault

FIFO (First-In First-Out):

- sostituisce pagina caricata da più tempo
- semplice ma può essere inefficiente
- non considera uso recente

LRU (Least Recently Used):

- sostituisce pagina usata meno di recente
- buone prestazioni ma costoso da implementare

Clock:

- variante semplificata di LRU
- pagine organizzate circolarmente
- bit utilizzo per ogni pagina
- lancetta cerca pagina con bit=0

```
Algoritmo Clock:
1. Se bit=0: sostituisci pagina
2. Se bit=1: azzera bit, avanza lancetta
3. Ripeti fino a trovare bit=0
```

Thrashing

Situazione critica dove sistema passa più tempo a gestire page fault che a eseguire processi.

Cause:

- troppi processi attivi
- troppo poche pagine per processo
- continuo swap-in/swap-out

Prevenzione:

- principio località temporale: dati usati recentemente saranno riusati presto
- principio località spaziale: dati vicini saranno acceduti insieme
- working set: insieme pagine attivamente usate da processo

Page Buffering

Tecnica per migliorare prestazioni gestione pagine.

Code buffer:

- **Free list**: pagine libere pronte per riassegnazione
- **Modified list**: pagine modificate da scrivere su disco

Vantaggi:

- riuso rapido pagine recenti
- scrittura aggregata su disco (più efficiente)
- riduce latenza page fault

Resident Set Management

Gestione numero pagine in RAM per processo.

Fixed Resident Set:

- numero fisso pagine per processo
- semplice ma rigido
- se troppo basso: molti page fault
- se troppo alto: spreco RAM

Variable Resident Set:

- numero pagine varia dinamicamente
- più efficiente ma overhead gestione
- adatta al working set del processo

Appendice: Tabelle Riassuntive

Confronto Preemptive vs Non-Preemptive

Caratteristica	Preemptive	Non-Preemptive
Interruzione processo	Sì	No
Controllo CPU	Scheduler	Processi
Tempi attesa	Minori per brevi	Maggiori per brevi
Overhead	Maggiore	Minore
Deadlock	Meno probabile	Più probabile
Starvation	Meno probabile	Più probabile
Throughput	Generalmente maggiore	Variabile

Confronto Algoritmi Scheduling

Algoritmo	Preemptive	Starvation	Overhead	Uso
FCFS	No	Possibile	Basso	Batch
SJF	No	Possibile	Medio	Batch
SRTN	Sì	Possibile	Alto	Batch

Algoritmo	Preemptive	Starvation	Overhead	Uso
RR	Sì	No	Medio	Interattivo
Priority	Variabile	Sì	Medio	Generale
HRRN	No	Ridotta	Medio	Interattivo

Confronto Algoritmi Sostituzione Pagine

Algoritmo	Complessità	Prestazioni	Implementazione
Optimal	Impossibile	Ottimale	Teorico
FIFO	Bassa	Mediocri	Semplice
LRU	Alta	Buone	Complessa
Clock	Media	Buone	Pratica

Note Finali

Questo manuale copre i principali argomenti del corso di Sistemi Operativi:

- Processi e Thread
- Concorrenza e Sincronizzazione

- Deadlock
- Inter-Process Communication
- Scheduling
- Gestione Memoria

Per approfondimenti consultare: + Documentazione POSIX per syscall Unix + Man pages per API specifiche + Testi di riferimento: Silberschatz, Tanenbaum, Stallings